PROJECT DOCUMENTATION

05

Deployment and Recovery Guide

How rishavprasad.in is built, deployed, verified, backed up and restored — with runbooks for the failures this stack can actually produce.

PREPARED FOR	SITE	VERSION	DATE
Rishav Prasad Owner and maintainer	rishavprasad.in Cloudflare Workers	1.0	31 October 2026

Contents

Deployment and Recovery Guide · version 1.0 · 14 sections

01 Deployment model 3

02 Prerequisites and credentials 3

03 First-time provisioning 4

04 Routine deployment 5

05 CI workflow reference 6

06 Configuration reference 6

07 Local development 7

08 Post-deploy verification 7

09 Backup strategy 8

10 Restore procedure 9

11 Rollback and recovery 9

12 Monitoring and logs 11

13 Running the test suites 12

14 Handover notes 12

01

Deployment model

The site is a Next.js static export attached to a Cloudflare Worker. Every deploy replaces both the Worker code and the static files it serves. Content lives separately in D1 and is snapshotted into the build.

Two ways a deploy starts

- **A push to `main`** — the normal code path. GitHub Actions typechecks, tests, builds and deploys.
- **A content publish** — the admin dashboard saves to D1, then asks GitHub to run the same workflow through a `repository_dispatch` event. The build fetches the new content before exporting.

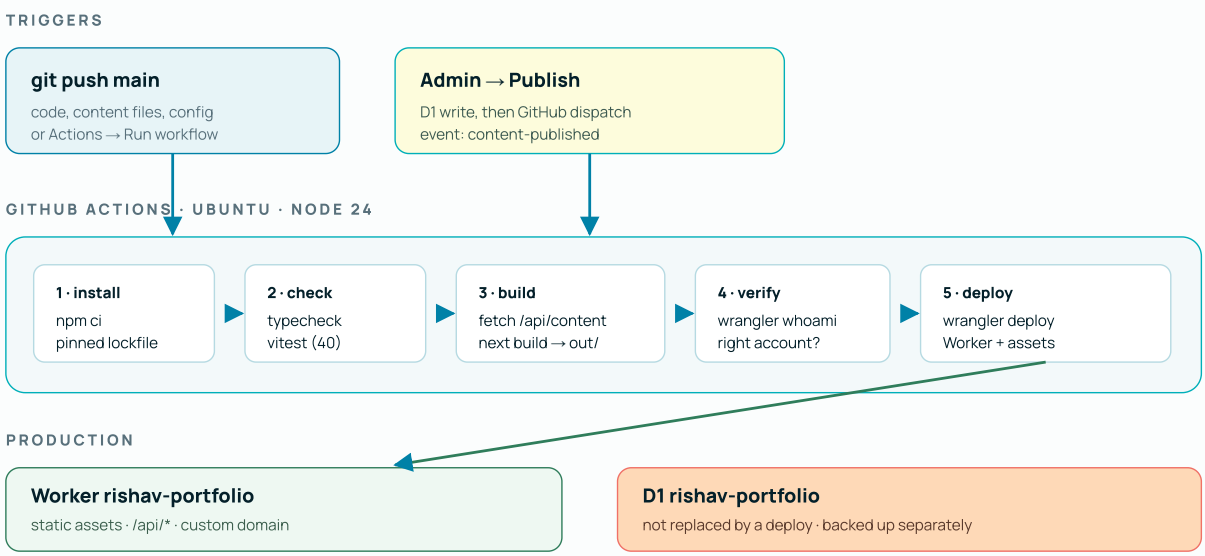


Figure 5.1 — Build and deployment paths. A deploy never carries database rows; D1 is versioned only by backups and migrations.

Static-first resilience. Cloudflare serves files from `out/` without invoking the Worker unless the path starts with `/api/` or `/media/`. If the Worker code errors, public pages normally continue to load; only API calls fail.

02

Prerequisites and credentials

REQUIREMENT	DETAIL	WHERE IT LIVES
Node.js	Version 24 is used in CI; 20+ is likely sufficient but was not tested below 24.	Developer machine / Actions runner
npm	Install with `npm ci` from the committed lockfile.	Repository
Wrangler	Pinned in `package-lock.json` (4.156.0 at handover).	Local `node_modules`
Cloudflare login	OAuth login for manual commands, or an API token for CI.	Local Wrangler config / GitHub secrets

REQUIREMENT	DETAIL	WHERE IT LIVES
GitHub access	Owner of `Rishav0523/rishav-portfolio`.	GitHub account
Domain	`rishavprasad.in` active in the same Cloudflare account.	Cloudflare dashboard / registrar
Worker secrets	`ADMIN_PASSWORD_HASH`, `SESSION_SECRET`, `TURNSTILE_SECRET`, `GITHUB_TOKEN`.	Cloudflare Worker settings

> **Never commit credentials.** The repository must not contain the admin password, the admin hash, the session secret, the Turnstile secret, the GitHub token or the Cloudflare API token. `.dev.vars` is git-ignored and exists only for local development.

03

First-time provisioning

These steps rebuild the environment from an empty Cloudflare account. They are recorded for disaster recovery and ownership transfer; the current environment is already provisioned.

1. **Clone and install.** `git clone https://github.com/Rishav0523/rishav-portfolio.git` then `npm ci`.
2. **Authenticate Wrangler.** `npx wrangler login` and confirm the account with `npx wrangler whoami`.
3. **Create the D1 database.** `npx wrangler d1 create rishav-portfolio`, then copy the returned `database_id` into `wrangler.jsonc`.
4. **Apply both migrations.** `npx wrangler d1 migrations apply rishav-portfolio --remote`. This creates the eight tables.
5. **Set the four Worker secrets** (next section). Do each one interactively; never pipe a command that might fail.
6. **Confirm variables.** `wrangler.jsonc` must contain `SITE_URL=https://rishavprasad.in` and `GITHUB_REPO=Rishav0523/rishav-portfolio`.
7. **Deploy.** `npm run build`, then `npx wrangler deploy`. Static assets upload with the Worker.
8. **Attach the domain.** The `routes` entry requests `rishavprasad.in` as a custom domain; accept it in the dashboard if prompted.
9. **Create the www redirect.** A Cloudflare Redirect Rule sends `www.rishavprasad.in` to the apex with a 301, preserving the query string. This is a dashboard rule, not a `_redirects` file.
10. **Set GitHub secrets** `CLOUDFLARE_API_TOKEN` and `CLOUDFLARE_ACCOUNT_ID`, then push to `main` and confirm a green workflow.

Generating and installing secrets

SECRET	HOW TO PRODUCE IT	INSTALL
`SESSION_SECRET`	32 random bytes, base64url encoded.	`npx wrangler secret put SESSION_SECRET`
`TURNSTILE_SECRET`	Copy the secret key of the Turnstile widget (not the public site key).	`npx wrangler secret put TURNSTILE_SECRET`
`ADMIN_PASSWORD_HASH`	`node scripts/hash-password.mjs "<password>"`; the output must start with `pbkdf2_sha256$100000$`.	`npx wrangler secret put ADMIN_PASSWORD_HASH`
`GITHUB_TOKEN`	Fine-grained token, repository `Rishav0523/rishav-portfolio`, Contents read and write, 90-day expiry.	`npx wrangler secret put GITHUB_TOKEN`

The empty-secret trap. Piping the hash script directly into `wrangler secret put` is unsafe: if the script refuses a short password, the pipe delivers nothing, Wrangler stores an empty value, and Cloudflare still lists the secret name. Run the script on its own, confirm the printed line, then paste it at the prompt.

04

Routine deployment

Deploy a code change

```
# from the repository root, on main
git pull
npm ci
npm run typecheck
npm test
npm run build          # runs the content fetch first
git push origin main   # GitHub Actions deploys
```

The CI workflow performs the same checks on a clean runner. Pushing is the normal delivery step; a manual `npx wrangler deploy` is for local experiments or emergency fixes.

Deploy a content change

1. Sign in at `https://rishavprasad.in/admin/`.
2. Edit the relevant tab and choose **Save changes**. This writes the content document to D1; the public site does not change yet.
3. Choose **Publish**. The Worker calls GitHub `repository_dispatch` with `event_type=content-published`.
4. Watch the `deploy` workflow finish, then hard-refresh the public page. The build fetched the updated document.

Why content needs a rebuild. Public pages are static HTML. The content API is a build-time input: `scripts/fetch-content.mjs` writes `content/published.json` before `next build`. Saving without publishing changes D1 and the admin preview but not the public HTML.

Schema changes

Add a numbered SQL file to `migrations/`, then apply it manually *before* deploying code that depends on it:

```
npx wrangler d1 migrations apply rishav-portfolio --remote
```

CI does not run migrations because the deploy token is scoped to Workers and static assets. The command is idempotent and skips already-applied files.

05

CI workflow reference

`.github/workflows/deploy.yml` runs on a push to `main`, a `repository_dispatch` of type `content-published`, and manual dispatch. A concurrency group cancels an in-progress run for the same ref.

STEP	WHAT IT DOES	FAILURE MEANING
`actions/checkout`, `setup-node`	Node 24 with npm cache.	Repository access or runner problem.
`npm ci`	Installs the exact lockfile.	Lockfile and `package.json` disagree.
`npm run typecheck`	App and Worker TypeScript projects.	A type error; nothing is deployed.
`npm test`	40 Vitest checks.	A unit regression; nothing is deployed.
`npm run build`	Fetches `/api/content` with `CONTENT_FETCH_STRICT=1`, then exports to `out/`.	The API is unreachable or the content fails validation. The build refuses to ship stale content.
`wrangler whoami`	Prints the authenticated account.	Bad or missing `CLOUDFLARE_API_TOKEN`.
`wrangler deploy`	Uploads Worker and assets.	Token lacks Workers Scripts edit or route access.
Failure summary	Appends the last Wrangler output to the run summary.	Read this first when a deploy fails with empty logs.

Empty step logs mean secrets, not code. If the workflow fails before any command output, check that `CLOUDFLARE_API_TOKEN` and `CLOUDFLARE_ACCOUNT_ID` still exist under Settings → Secrets and variables → Actions.

06

Configuration reference

NAME	KIND	VALUE / PURPOSE
`SITE_URL`	Worker var + build env	`https://rishavprasad.in`; canonical URLs, sitemap and the build-time fetch base.
`GITHUB_REPO`	Worker var	`Rishav0523/rishav-portfolio`; target of the publish dispatch.
`CONTENT_FETCH_STRICT`	Build env	`1` in CI; makes an unreachable content API fail the build.
`ADMIN_PASSWORD_HASH`	Secret	PBKDF2-SHA256 at exactly 100,000 iterations.
`SESSION_SECRET`	Secret	HMAC key for the 12-hour admin cookie.
`TURNSTILE_SECRET`	Secret	Server-side Turnstile verification.
`GITHUB_TOKEN`	Secret	Fine-grained token used only for repository dispatch.

NAME	KIND	VALUE / PURPOSE
`MEDIA`	R2 binding	Commented out; intentionally absent until R2 is enabled.

There is no production `.env` file. Worker variables live in `wrangler.jsonc`, secrets in Cloudflare, and CI values in the workflow or repository settings.

07

Local development

The local stack serves the last build in `out/` and a local D1 database. Manage it carefully: Wrangler reads `out/` only at startup, and orphaned `workerd` processes keep the local database locked.

```
# once: create the local schema
npx wrangler d1 migrations apply rishav-portfolio --local

# build, then start exactly one server
npm run build
npx wrangler dev --port 8787
```

`.dev.vars` (git-ignored) supplies the local secrets:

```
TURNSTILE_SECRET=...
SESSION_SECRET=...
ADMIN_PASSWORD_HASH=pbkdf2_sha256$100000$...
```

Two failure modes that waste hours. (1) Stopping the Wrangler task can leave orphaned `workerd.exe` children holding port 8787 and the SQLite file; kill the whole process tree, not one PID, and confirm the port is free. (2) Rebuilding while the server runs serves stale chunks and produces `ChunkLoadError` or HTML in place of JavaScript; stop the server before every rebuild.

Turnstile widgets are bound to hostnames. Use `http://localhost:8787` for the widget; `127.0.0.1` is not an allowed hostname in the widget configuration.

08

Post-deploy verification

Run these checks after every production deploy. They take about two minutes and catch the failures this stack has actually produced.

CHECK	COMMAND / ACTION	EXPECTED
Health and database	`curl -s https://rishavprasad.in/api/health`	`{"ok":true,"service":"portfolio-api","database":"ok"}`
Home page	`curl -s -o /dev/null -w "%{http_code}" https://rishavprasad.in/`	`200`
Unknown path	`curl -s -o /dev/null -w "%{http_code}" https://rishavprasad.in/nope/`	`404`
Content API	`curl -s https://rishavprasad.in/api/content | head -c 80`	JSON beginning with `{"version":1`

CHECK	COMMAND / ACTION	EXPECTED
Admin gate	`curl -s -o /dev/null -w "%{http_code}" https://rishavprasad.in/api/admin/content`	`401`
R2 state	`curl -s -o /dev/null -w "%{http_code}" https://rishavprasad.in/media/x`	`503` while R2 is disabled
Public routes	`E2E_BASE=https://rishavprasad.in node tests/e2e.mjs`	31 passed, 0 failed
Responsive sweep	`E2E_BASE=https://rishavprasad.in node tests/e2e-responsive.mjs`	84 passed, 0 failed
AirShare	`E2E_BASE=https://rishavprasad.in node tests/e2e-airshare.mjs`	11 passed, 0 failed
Admin sign-in	Open `/admin/` and sign in	Dashboard loads; Save and Publish work

09

Backup strategy

D1 holds the only non-reproducible data: contact messages, the content document, login throttling rows and AirShare signalling rows. The code and public content are in git; the database is not.

What was produced at handover

FILE	SIZE	CONTENTS	COMMITTED?
`backups/rishav-portfolio-2026-10-31.sql`	28.4 KB	Full export: schema, migrations, content and all personal rows.	No — git-ignored
`backups/schema-and-content-2026-10-31.sql`	11.3 KB	Schema, migration history and the public content row only.	Yes

The sanitized file was verified by executing it into an empty SQLite database; it restores one `site_content` row of 8,767 bytes. SHA-256 of the committed file: `fc594cd9…c171d77b`.

Why the full backup is not in GitHub. It contains a real contact message, its sender's name and email, and login/AirShare metadata. Committing it to a public repository would publish personal data. Keep the full export on encrypted local storage or a private drive, never in the repo.

Producing a new backup

```
# Full export — personal data inside; store privately
npx wrangler d1 export rishav-portfolio --remote \
  --output backups/rishav-portfolio-YYYY-MM-DD.sql

# Check it is not empty and contains the tables
grep -c "INSERT INTO" backups/rishav-portfolio-YYYY-MM-DD.sql
grep -oE "INSERT INTO \"[a-z_]+\"" backups/rishav-portfolio-YYYY-MM-DD.sql | sort -u
```

Export briefly affects the database. Wrangler warns that queries may be unavailable while the export runs. It took seconds for this 94 kB database. Run exports at a quiet time and avoid doing one during a deploy or a content edit.

Recommended schedule

- **Before** any migration, bulk content change or secret rotation.
- **Monthly** if contact messages matter; the content document is also recoverable from a git build snapshot.
- **After** any unexplained data loss or a suspicious admin session.
- Keep at least three generations, one of them off the machine.

10

Restore procedure

There are two restore scenarios. Choose deliberately; the full export overwrites rows and can conflict with data created after the backup.

Restore the public content only

The sanitized backup is safe for a fresh database or for repairing one content row. To restore the document into the current database without touching messages:

```
npx wrangler d1 execute rishav-portfolio --remote \
  --file backups/schema-and-content-2026-10-31.sql
```

If the database already contains the same row, SQLite will reject the duplicate primary key. In that case, extract just the `site_content` statement into a temporary file, or update the row through the admin dashboard, then publish.

Rebuild the database from a full export

1. Download the chosen export and confirm it is the intended date.
2. Create a new database: `npx wrangler d1 create rishav-portfolio-restore`.
3. Import: `npx wrangler d1 execute rishav-portfolio-restore --remote --file backups/rishav-portfolio-YYYY-MM-DD.sql`.
4. Spot-check: `SELECT COUNT(*) FROM site_content;` and `SELECT COUNT(*) FROM contact_messages;` on the new database.
5. Point the `DB` binding in `wrangler.jsonc` at the new `database_id`, deploy, and verify `/api/health`.
6. Keep the old database until the new one has served correctly for a day, then delete it deliberately.

> **Never import a full export into a populated database as a shortcut.** Duplicate primary keys and `UNIQUE` constraints will abort partway, and you may not know which rows landed. Restore into a new database and switch the binding, or restore only the statements you need.

Verify a backup without touching production

The sanitized file can be checked in any SQLite client. The handover check executed the script into an in-memory database and confirmed one content row of 8,767 bytes. Do the same for a new export before trusting it.

11

Rollback and recovery

Roll back a bad deploy

```
npx wrangler deployments list --name rishav-portfolio
npx wrangler rollback <deployment-or-version-id>
```

A rollback restores the Worker code and the static assets attached to that version. It does **not** roll back D1. If a content edit caused the problem, fix the content and publish again; rollback does not revert the database.

Runbook: site returns errors or is blank

1. Check `https://rishavprasad.in/api/health` and the home page separately. If the home page is fine but the API fails, the issue is the Worker or D1, not the assets.
2. Open Cloudflare → Workers → `rishav-portfolio` → Deployments and confirm the expected version is at 100%.
3. Inspect logs with `npx wrangler tail rishav-portfolio` while reproducing the request.
4. If the latest deploy is at fault, roll back to the previous deployment id.
5. If the rollback page is also wrong, re-deploy the last known-good commit from GitHub Actions.

Runbook: API returns 503

SYMPTOM	LIKELY CAUSE	ACTION
Health says `database:"unavailable"`	D1 binding or database outage.	Confirm the binding id in `wrangler.jsonc`; check D1 status in the dashboard; retry.
Admin says "not configured"	`ADMIN_PASSWORD_HASH` or `SESSION_SECRET` missing or empty.	Reinstall the secrets interactively; no redeploy needed.
Contact says "not configured"	`TURNSTILE_SECRET` missing.	Reinstall the Turnstile secret.
Publish says "not configured"	`GITHUB_TOKEN` missing or expired.	Create a new fine-grained token and replace the secret.
Media returns 503	R2 disabled or binding commented out.	Expected state; enable R2 only if uploads are wanted.
Everything returns a Worker error page	Unhandled exception in the Worker.	Tail the logs, roll back, then fix forward.

Runbook: admin lockout

1. Generate a new hash with `node scripts/hash-password.mjs "<new long password>"`; confirm it starts with `pbkdf2_sha256$100000$`.
2. Run `npx wrangler secret put ADMIN_PASSWORD_HASH` and paste the line. No redeploy is required.
3. If sessions must be invalidated too, rotate `SESSION_SECRET`; every admin is signed out immediately.
4. If login still returns 429, wait 15 minutes for the per-IP attempt window to clear.
5. If login returns 503, the environment reports a missing secret; list names with `npx wrangler secret list`.

Runbook: lost repository or machine

1. Clone `https://github.com/Rishav0523/rishav-portfolio.git` on a new machine.
2. `npm ci` and restore `.dev.vars` from the password manager if local development is needed.
3. If GitHub is also lost, restore from a local clone and re-create the repository; update `GITHUB_REPO` and the `GITHUB_TOKEN` secret.
4. Restore D1 from the latest full export into a new database as described above.
5. Re-add the two GitHub Actions secrets and run the workflow manually.

Runbook: domain, DNS or certificate

- **Nameservers changed?** The zone stops resolving if the registrar no longer delegates to `alla` and `koa.ns.cloudflare.com`. Restore the delegation in the registrar.
- **Certificate expired?** Cloudflare-managed certificates renew automatically; if the certificate shows expired, check the custom domain status and re-issue it from SSL/TLS → Edge Certificates.
- **www stopped redirecting?** The Redirect Rule is dashboard state, not repository state. Recreate it as a 301 to `https://rishavprasad.in` preserving the query string.
- **DNSSEC?** It is currently unsigned. If you enable it later and then change registrars, remove the DS record first or the domain can stop resolving.

12

Monitoring and logs

WHAT	HOW TO SEE IT	RETENTION
Worker exceptions	`npx wrangler tail rishav-portfolio`	Live stream only
Deploy history	`npx wrangler deployments list` or the dashboard	Cloudflare account history
CI results	GitHub Actions tab	GitHub defaults
D1 usage	`npx wrangler d1 info rishav-portfolio`	Rolling 24-hour metrics
Contact messages	Admin → Messages	Until deleted
Uptime / errors	No external monitor configured	—

For a personal site this is proportionate, but nothing will alert the owner when the site is down. A free uptime checker on `/api/health` is the smallest useful addition if that matters.

13

Running the test suites

These commands reproduce the results in document 04. No local server is needed for the production runs.

```
# static checks
npm run typecheck
npm test

# production browser suites
E2E_BASE=https://rishavprasad.in node tests/e2e.mjs
E2E_BASE=https://rishavprasad.in node tests/e2e-responsive.mjs
E2E_BASE=https://rishavprasad.in node tests/e2e-airshare.mjs

# legacy Node LAN server
node --test airshare/test/airshare.test.js
```

COMMAND	EXPECTED	NOTES
`npm test`	40 passed	3 suites; no network.
`node --test airshare/test/airshare.test.js`	8 passed	Starts a local server on an ephemeral port.
`node tests/e2e.mjs`	31 passed	Set `E2E_BASE` to production; admin sign-in is skipped without a local password.
`node tests/e2e-responsive.mjs`	84 passed	77 route/width checks plus 7 page-error checks.
`node tests/e2e-airshare.mjs`	11 passed	Creates a transient room; rows expire.

> **AirShare tests write to production D1.** They create presence, signal and join-attempt rows through the public API. The rows expire by TTL; the join counter persists as a rate-limit record. Run them when a few extra join attempts will not matter.

14

Handover notes

- Everything runs on free-tier Cloudflare and GitHub services; there is no paid dependency in the build or runtime.
- The owner's GitHub account, Cloudflare account and registrar control the site. There is no shared or delegated account.
- The full database export is the only artifact that must not be published. Store it privately and rotate it after any contact message you would not want exposed.
- Always Use HTTPS, DNSSEC and the stray secret are dashboard actions tracked in documents 03, 04 and 07.